RecoMart Pipeline - Demo Video Narration Script

Duration: 5-10 minutes | Read this during your demo video walkthrough

---

OPENING (30 seconds)

> *"Hello everyone. Today I'll be walking you through our end-to-end data management pipeline built for **RecoMart**, an e-commerce startup that wants to build a **data-driven recommendation engine** to enhance customer engagement and cross-selling opportunities.*
>
> *Our pipeline is designed to continuously ingest, validate, transform, and train models on incoming data - aligned with modern data stack practices and ML workflows.*
>
> *The pipeline covers **all 10 stages** from data ingestion to model training, fully orchestrated using **Prefect**, with experiment tracking via **MLflow** and feature management via **Feast**."*

STAGE 0: PROJECT STRUCTURE (30 seconds)

> *"Let me start by showing the project structure. Our codebase is fully modular, organized into clear packages:*
>
> *- **`config/`** - Central YAML configuration for all pipeline settings*
>
> *- **`src/ingestion/`** - Data collection from two sources*
>
> *- **`src/validation/`** - Data quality checks and reporting*
>
> *- **`src/preparation/`** - Cleaning, encoding, normalization, and EDA*
>
> *- **`src/transformation/`** - Feature engineering*
>
> *- **`src/feature_store/`** - Feast integration for feature management*
>
> *- **`src/versioning/`** - Hash-based data versioning and lineage*
>
> *- **`src/training/`** - Model training with MLflow*
>
> *- **`orchestration/`** - Prefect flow definition*
>
> *Everything is configurable through `pipeline_config.yaml` - paths, thresholds, model hyperparameters, and MLflow settings are all centrally managed."*

STAGE 1: DATA INGESTION (1 minute)

> *"The first stage handles **data collection from two distinct sources**, as required by the assignment.*
>
> *__Source 1 - CSV Ingestion (Kaggle Dataset):__ We use `kagglehub` to programmatically download the **Ecommerce Product Recommendation Dataset** by Kartikey Bartwal. This dataset contains **1,018 product records** with **13 features** including: number of clicks on similar products, number of purchases, average rating, gender, price, brand, customer sentiment score, holiday and seasonal flags, geographical*

*location, and the target variable - **probability for the product to be recommended**.*

***Source 2 - Simulated REST API (User Interactions):** We generate **5,000 synthetic user interaction records** for **200 users**, simulating what would come from a live API. Each interaction has a user ID, product ID, action type - which can be view, click, purchase, or rate - a rating, and a timestamp spanning the last 90 days.*

*Both ingestion scripts implement **retry logic with exponential backoff**, **error handling**, and **detailed logging**. The raw data is stored in a structured data lake layout, partitioned by source type and date - for example: `data/raw/kaggle/product_recommendations/2026-07-19/`.*

Let me show you the ingestion logs..."

[Show the logs/pipeline.log file or terminal output showing ingestion success]

[OK] STAGE 2: DATA VALIDATION (1 minute)

*"Stage 2 applies **automated data quality checks** on both datasets.*

*For the **product data**, we validate:*

- Schema completeness - ensuring all expected columns are present

- Missing value percentages - with configurable thresholds

- Duplicate row detection

- Range checks - ratings between 0-5, prices non-negative

- Data type consistency

*For the **user interaction data**, we validate:*

- Schema - user_id, product_id, rating, timestamp, action_type

- Rating range - 1 to 5 for rated interactions

- Valid action types - must be view, click, purchase, or rate

- Timestamp format - must be parseable as datetime

- Duplicate interaction checks

*The validator generates an **HTML Data Quality Report** with a visual pass/fail table and an overall quality score. Let me show you the report..."*

[Open `reports/data_quality_report.html` in browser]

"As you can see, our data quality score is displayed at the top, with each individual check showing pass or fail status along with details. Clean, validated data is then saved to the `data/validated/` directory."

STAGE 3: DATA PREPARATION & EDA (1 minute)

*"In Stage 3, we perform **data cleaning, preprocessing, and exploratory data analysis**.*

***Cleaning:** Missing numerical values are imputed with the median, and categorical missing values with the*

mode. Any remaining null rows are dropped.

***Encoding:** Categorical features like Gender, Brand, Holiday, Season, and Geographical Location are transformed using **Label Encoding**.*

***Normalization:** Numerical features including prices, ratings, sentiment scores, and click counts are scaled using **MinMaxScaler** to bring them into the 0-1 range. The encoder and scaler objects are saved for later inverse transformation.*

***EDA:** We generate **6 visualizations** saved to `reports/eda_plots/`:*

*1. **Rating distribution histogram** - shows the spread of product ratings*

*2. **Price distribution box plot** - reveals pricing outliers*

*3. **Brand frequency bar chart** - top 15 most common brands*

*4. **Correlation heatmap** - relationships between all numerical features*

*5. **User activity distribution** - how many interactions per user*

*6. **Item popularity distribution** - how many interactions per product*

Let me show you some of these plots..."

[Open the EDA plots from reports/eda_plots/]

STAGE 4: FEATURE ENGINEERING (1 minute)

*"Stage 4 creates **engineered features** optimized for recommendation algorithms.*

***Product Features** derived from the Kaggle dataset:*

*- **Click-to-purchase ratio** - clicks divided by purchases plus one, indicating conversion potential*

*- **Price bracket** - quartile-based categorization into Low, Medium, High, and Premium*

*- **Brand popularity** - normalized count of products per brand*

*- **Quality score** - weighted combination of product rating (70%) and sentiment score (30%)*

*- **Seasonal demand** - interaction of season and holiday flags*

***User Features** derived from interaction data:*

*- **Activity frequency** - total interactions per user*

*- **Average rating given** - mean rating for rated interactions*

*- **Rating variance** - standard deviation of ratings per user*

*- **Unique items interacted** - count of distinct products*

*- **Purchase ratio** - purchases divided by total interactions*

*We also build a **user-item interaction matrix** in long format for collaborative filtering.*

*All features are stored in **three formats**: CSV for readability, **Parquet for Feast compatibility**, and **SQLite** in our data warehouse at `data/warehouse.db`."*

STAGE 5: FEATURE STORE - FEAST (45 seconds)

> *"Stage 5 implements a **feature store using Feast**, the open-source feature store framework.*
>
> *We define two **entities**: `product_id` and `user_id`.*
>
> *We create two **Feature Views**:*
>
> *- **`product_features`** with 7 fields: click-to-purchase ratio, price bracket, brand popularity, quality score, rating, sentiment score, and price*
>
> *- **`user_features`** with 5 fields: activity frequency, average rating given, rating variance, unique items interacted, and purchase ratio*
>
> *Feast is configured with a **local provider**, using a **file-based offline store** pointing to our Parquet files and a **SQLite online store** for low-latency feature serving.*
>
> *The `feast apply` command registers our feature definitions, and `materialize_incremental` pushes data from the offline store to the online store for inference.*
>
> *This enables **versioned feature retrieval** for both model training and real-time inference."*

STAGE 6: DATA VERSIONING & LINEAGE (45 seconds)

> *"Stage 6 tracks **data versions and lineage** across the pipeline.*
>
> *For every data file produced by the pipeline - from raw ingested data through validated, prepared, and engineered features - we compute a **SHA-256 hash** and record metadata including: filename, hash, file size, row count, column count, source, transformation applied, version number, and timestamp.*
>
> *We also build a **lineage graph** as JSON, mapping the transformation flow: raw data validated data prepared data features. This provides full **data provenance** - we can trace any feature value back to its original source record.*
>
> *All metadata is stored in `data/versioning/versions.json` and `data/versioning/lineage.json`, and a summary table is printed to the console for quick inspection."*

STAGE 7: MODEL TRAINING & EVALUATION (1.5 minutes)

> *"Stage 7 trains **two recommendation models** and tracks everything with **MLflow**.*
>
> ***Model 1 - Content-Based Filtering:***
>
> *We train a **Gradient Boosting Regressor** using the product features to predict the recommendation probability. With **15 input features** and **1,018 samples**, after an 80/20 train-test split, we achieve:*
>
> *- **RMSE: 0.0769** - very low prediction error*
>
> *- **MAE: 0.0454** - average absolute error under 5%*
>
> *- **R^2: 0.9361** - the model explains 93.6% of the variance, which is excellent*
>
> ***Model 2 - Collaborative Filtering (SVD):***
>
> *We train a **Singular Value Decomposition** model from the Surprise library on the user-item interaction*

*matrix. Using **1,211 rated interactions** with 3-fold cross-validation:*

*- **RMSE: 1.0676***

*- **MAE: 0.8705***

*- We also compute **Precision@10, Recall@10, and NDCG@10** ranking metrics*

***MLflow Integration:** All experiments are tracked in a **SQLite-backed MLflow** instance. For each run, we log:*

*- **Parameters**: algorithm type, n_estimators, max_depth, n_factors, n_epochs*

*- **Metrics**: RMSE, MAE, R^2, Precision@K, Recall@K, NDCG@K*

*- **Artifacts**: The saved model files*

Both trained models are saved - the content-based model as a joblib file and the SVD model as a pickle file - in the `models/` directory."

PIPELINE ORCHESTRATION - PREFECT (45 seconds)

*"The entire pipeline is orchestrated using **Prefect**, a modern workflow orchestration framework.*

*We define **7 Prefect tasks**, each wrapping one pipeline stage:*

1. Data Ingestion (with 2 automatic retries)

2. Data Validation

3. Data Preparation

4. Feature Engineering

5. Feature Store

6. Data Versioning

7. Model Training

*These are chained together in a **Prefect flow** called `RecoMart Recommendation Pipeline`. Data flows from one task to the next - the output of ingestion feeds into validation, validation feeds into preparation, and so on.*

*Prefect provides **task-level logging**, **automatic retries** on failure, **run tracking**, and a web dashboard for monitoring. Each task reports its status with clear success or failure indicators."*

OUTPUT SUMMARY (30 seconds)

"Let me quickly summarize all the outputs generated by our pipeline:

*- **Raw data** - Kaggle CSV + synthetic interactions, partitioned by date*

*- **Validated data** - cleaned datasets in `data/validated/`*

*- **HTML quality report** - comprehensive pass/fail metrics*

*- **6 EDA plots** - rating, price, brand, correlation, user activity, item popularity*

*- **Engineered features** - CSV, Parquet, and SQLite formats*

*- **Feast feature store** - registered entities and feature views with online store*

*- **Data versioning** - SHA-256 hashes, version history, and lineage graph*

*- **2 trained models** - content-based and collaborative filtering*

*- **MLflow experiment tracking** - parameters, metrics, and model artifacts*

*- **Complete pipeline logs** - timestamped records of every operation"*

CLOSING (15 seconds)

*"In summary, we've built a **production-ready, end-to-end data management pipeline** that takes raw e-commerce data from multiple sources and transforms it into trained recommendation models - all automated, logged, versioned, and orchestrated.*

The pipeline is fully modular, configurable, and extensible - making it easy to add new data sources, features, or models in the future.

Thank you for watching."

Key Metrics to Highlight During Demo

Model	Metric	Value
Content-Based (GBR)	RMSE	**0.0769**
Content-Based (GBR)	MAE	**0.0454**
Content-Based (GBR)	R^2	**0.9361**
Collaborative (SVD)	RMSE	**1.0676**
Collaborative (SVD)	MAE	**0.8705**

What to Show on Screen

1. **Project Structure** VS Code file explorer
2. **Terminal** `python run_pipeline.py` output (all 7 stages)
3. **Data Quality Report** Open `reports/data_quality_report.html` in browser
4. **EDA Plots** Open images from `reports/eda_plots/`
5. **Versioning Metadata** Open `data/versioning/versions.json`
6. **Lineage Graph** Open `data/versioning/lineage.json`
7. **MLflow DB** Show that `mlflow.db` exists (or run `mlflow ui`)
8. **Models** Show `models/` directory with saved files
9. **Logs** Show `logs/pipeline.log` with timestamped entries